# Personal Assistant System

Product & Technical Requirements

v0.1 — April 22, 2026

## 1. Overview

### 1.1 Goal

Build a personal assistant composed of many small, independent agents that share a common infrastructure. Start with one working agent today (Email PM) and add agents incrementally as life demands them. Every new agent should reuse existing channels, tools, and resources — the cost and risk of adding an agent should decrease over time, not increase.

### 1.2 Guiding principles

- **Small agents, big system.** Prefer many focused agents over one monolithic one. Each agent owns one concern: one project, one relationship, one channel's triage. Easier to write, easier to trust, easier to turn off.
- **Channels are pipes, not agents.** Email, WhatsApp, Calendar expose two things: an event stream (things arrived) and an action API (do things). No intelligence in the channel layer.
- **Resources are managed, not embedded.** LLMs, storage, and observability sit underneath as swappable services. Agents reference them by logical name, not provider.
- **Cheap when possible, capable when needed.** Local model (Qwen3-30B on the 3090) for the majority of work; Claude API when the agent itself decides the task warrants it. Model choice is an explicit agent decision, not hidden routing magic.
- **Human-in-the-loop by default.** No autonomous sending in v0.1. Agents draft, label, and flag. Sol reviews and dispatches.
- **Boundaries enable upgrades.** Every layer can be swapped without touching the others. Today's simple Postgres queue can become NATS without touching agents; Ollama can become vLLM without touching anything but the model router.

### 1.3 Non-goals for v0.1

- No autonomous send on any channel. Drafts only.
- No orchestrator agent. Agents subscribe to events directly.
- No interface agent with voice/chat UX. Sol interacts via Gmail (for now) and CLI.
- No Calendar, Slack, Notion, or other channels beyond Gmail + WhatsApp.
- No multi-user support. Single-tenant, one person's system.
- No cloud deployment. Runs on Sol's hardware (3090 host + Docker).

# 2. Product requirements

## 2.1 User (Sol)

One user. Technical. Wants the system to start simple and grow. Values the ability to inspect, modify, and swap any layer. Primary pain: time spent on inbox triage and followups across multiple Gmail accounts and WhatsApp numbers.

## 2.2 Accounts in scope for v0.1

Multi-account is first-class from day 1. Sol has multiple Gmail accounts (personal + businesses including Atheer and potentially Accenture workspace) and two WhatsApp numbers. Every event and action must carry an explicit account identifier.

Examples of account IDs used throughout this document: gmail.personal, gmail.atheer, wa.main, wa.secondary. These are stable slugs stored in a channel_accounts table, not hardcoded.

## 2.3 Use cases for v0.1

### UC-1: Inbox triage

**Trigger:** new email arrives in any configured Gmail account.

**Behavior:** Email PM agent reads the message, classifies it, applies a Gmail label (pm/urgent, pm/waiting-on-them, pm/waiting-on-me, pm/fyi, pm/done), and optionally drafts a reply if the message is routine.

**Success:** unread inbox is always labeled within minutes; Sol can filter by label to see what actually needs attention.

### UC-2: Followup tracking

**Trigger:** email sent or received where a response or action is expected within a deadline.

**Behavior:** Email PM agent logs the followup (thread_id, who owes what, deadline). Daily cron surfaces overdue followups via Gmail labels (later: Telegram).

**Success:** nothing important falls through the cracks; Sol sees overdue items once a day.

### UC-3: WhatsApp event capture

**Trigger:** message arrives on either WhatsApp number.

**Behavior (v0.1):** event is published to the bus and logged. No agent acts on WhatsApp yet — this use case exists to prove the pipe works end-to-end.

**Success:** within v0.2, a WhatsApp agent can be added without any changes to the adapter.

## 2.4 Out of scope use cases (named for future)

- UC-F1: Per-project subject agent (e.g. Berwyn Heights deal) — v0.2
- UC-F2: Interface agent via Telegram for Q&A and approvals — v0.2
- UC-F3: Calendar triage and scheduling — v0.3
- UC-F4: Cross-channel subject tracking (same project via email + WhatsApp) — v0.3

- UC-F5: Voice interface — later

### 2.5 Quality bar

- Correctness: agent never sends a message autonomously. Drafts only.
- Privacy: all data stays on Sol's hardware except LLM calls to Anthropic API when explicitly escalated.
- Latency: email arriving → labeled in Gmail within 60 seconds for 95% of messages.
- Observability: every agent invocation traceable in Langfuse within 5 seconds of execution.
- Cost: local model handles ≥80% of agent calls; Claude API spend under $30/month for the single-agent v0.1.

## 3. Architecture

### 3.1 Layering

The system has five logical layers. Each layer has a narrow contract with the layers above and below it, and can be replaced without touching the others.

| Layer | Purpose | Components (v0.1) |
|---|---|---|
| 1. Human interface | How Sol interacts with the system and approves actions | Gmail inbox (drafts + labels), CLI |
| 2. Agents | Independent units that do work | Email PM (more later) |
| 3. Agent runtime | Lifecycle, scheduling, dispatch | Single Python process with registry.yaml |
| 4. Tools & channels | Events in, actions out | Gmail adapter, WhatsApp adapter, LangChain toolkits |
| 5. Resources | Shared managed services | Ollama, LiteLLM (later), Postgres, Langfuse |

### 3.2 Event bus (v0.1: Postgres queue)

Channels publish events. Agents subscribe to events. Neither knows about the other. The bus is the one mandatory shared contract in the system.

v0.1 uses Postgres as the bus: an events table plus a subscriptions mechanism. Justification: Sol's system is single-node, agents are few, and Postgres is already required for LangGraph checkpoints and followup state. Adding NATS today is one more container with no marginal benefit.

Upgrade path to NATS JetStream: the events table schema matches the NATS subject/payload convention exactly. Migrating means changing the adapter's publish() implementation and the runtime's subscribe() implementation — agent code is untouched.

### 3.3 Multi-account model

Every channel has N accounts. Every account is one row in channel_accounts. Adapters are multi-tenant — one gmail-adapter process handles all Gmail accounts by spinning up one worker per row.

Event topics encode the account: email.received.gmail.atheer, whatsapp.message.received.wa.main. The account_id is also a top-level field in the event payload — never rely solely on topic parsing.

Tools are account-parameterized. Instead of binding a Gmail toolkit to one account at agent creation, tools take account_id as an argument and the runtime resolves credentials per call. This lets one agent act across multiple accounts when needed, while per-account agents remain possible via subscription filtering.

### 3.4 Separation of concerns: subject vs channel

Two axes that were initially confused and are now separated:

- **Channel concerns** (Gmail, WhatsApp, Calendar): stable, few, mostly stateless. Adapters handle auth, fetching, sending. No LLM. Grow slowly — a new channel is an infrequent event.
- **Subject concerns** (Berwyn deal, AI startup, partner, each significant project/relationship): dynamic, many, long-lived memory. Agents accumulate context across channels. New subject agents are created regularly — this is where the 'agent creator' UX matters.

v0.1 has no subject agents yet — Email PM is a channel-flavored agent (triage, not subject state). Subject agents arrive in v0.2.

## 4. Components

### 4.1 Shared package

**Purpose:** single source of truth for schemas, DB helpers, and cross-cutting utilities.

**Language:** Python 3.12.

**Key modules:**

- events.py — Pydantic models: EmailReceivedEvent, WhatsAppMessageEvent, BaseEvent.
- approvals.py — PendingApproval model.
- accounts.py — ChannelAccount model and lookup helpers.
- db.py — Postgres connection pool helper.
- bus.py — publish(event) / subscribe(pattern, handler) abstraction. v0.1 implementation hits Postgres; v0.2 can swap to NATS with no caller changes.

### 4.2 Gmail adapter

**Language:** Python (FastAPI).

**Responsibilities:**

- On startup: load all active Gmail accounts from channel_accounts.
- For each account: run users.watch() to register with a Google Pub/Sub topic, persist history_id cursor.
- Cron: renew watch every 24 hours (Google expires watches at 7 days).
- POST /gmail/push endpoint: receives Pub/Sub push notifications, extracts emailAddress, looks up account, calls users.history.list(startHistoryId=cursor), fetches new messages, publishes email.received.gmail.<account_slug> events to the bus, updates cursor.
- Add-account CLI: python -m channels.gmail.add_account --id gmail.atheer --name 'Atheer' — runs OAuth, stores credentials, calls watch().

**Actions:** No direct action API. Agents use LangChain's GmailToolkit directly, which reads credentials from channel_accounts via a tool factory.

## 4.3 WhatsApp adapter

**Language:** Node.js (Baileys + Fastify).

**Responsibilities:**

- On startup: load active WhatsApp accounts, start one Baileys socket per account with its own auth state directory.
- On messages.upsert event from any socket: format to WhatsAppMessageEvent schema, publish whatsapp.message.received.<account_slug> to bus.
- POST /whatsapp/send { account_id, chat_id, text }: dispatch to the correct socket's sendMessage.
- Add-account CLI: node src/add_account.js --id wa.main — creates auth dir, prints QR, waits for scan, writes row to channel_accounts.

**Safety:** Baileys is an unofficial WhatsApp client. Risk of account ban is low for single-user non-spam usage but non-zero. Sol accepts this risk for personal WhatsApp numbers. For any business number, migrate to Cloud API later.

## 4.4 Agent runtime

**Language:** Python.

**Responsibilities:**

- Read registry.yaml to discover configured agents and their subscriptions/schedules.
- For each event-driven agent: run shared.bus.subscribe(pattern, handler) where handler instantiates the agent's LangGraph graph and invokes it with the event.
- For each scheduled agent: run cron-like loop invoking the agent on schedule (daily, hourly, etc.).
- Crash isolation: one agent error does not take down others. Exceptions are logged to Langfuse and the event is marked failed (retriable).
- Account-scoped tool factory: get_gmail_tools(account_id) returns a LangChain GmailToolkit bound to that account's credentials.

**v0.1 implementation:** single async Python process. Adequate for single-user scale. Upgrade path: each agent can move to its own process (systemd unit, separate container) when resource isolation becomes necessary.

## 4.5 Email PM agent

**Framework:** LangGraph create_agent with a Postgres checkpointer.

**Default model:** local/qwen3-30b (via Ollama). Sufficient for categorization, labeling, and routine drafts.

**Escalation:** explicit tool escalate_to_claude(reason, task) routes specific sub-tasks to Claude Opus 4.7 via Anthropic API. Used when agent identifies sensitive or complex drafting.

**Subscribes to:** email.received.gmail.*

**Schedule:** daily tick at 8am local for overdue followup review.

### Tools (bound)

- gmail_search(account_id, query) — via LangChain GmailToolkit
- gmail_get_thread(account_id, thread_id)
- gmail_create_draft(account_id, thread_id, body) — draft only, never send
- gmail_add_label(account_id, thread_id, label)
- log_followup(account_id, thread_id, direction, deadline, note) — direction is 'waiting_on_them' or 'waiting_on_me'
- list_overdue_followups() — returns followups past their deadline
- categorize_thread(account_id, thread_id, category) — applies one of pm/urgent, pm/waiting-on-them, pm/waiting-on-me, pm/fyi, pm/done
- escalate_to_claude(reason, task) — returns text produced by Claude for inclusion in a draft

### Hard constraints (enforced by not binding the tool)

- Agent CANNOT send email. No gmail_send tool exists in its tool list.
- Agent CANNOT delete email. No gmail_delete tool in its tool list.
- Agent CANNOT cross accounts in a single reply. System prompt enforces 'reply from the account that received the message'; drafts always go back to the receiving account.

## 4.6 Resources

### Ollama

**Deployment:** runs directly on the host (not Docker) to avoid GPU passthrough overhead. One 3090 dedicated.

**Model:** qwen3:30b-a3b-instruct. MoE architecture, ~3B active parameters, strong tool-use performance, fits comfortably on 24GB VRAM.

**Access:** agents reach it at http://host.docker.internal:11434 from inside containers.

### Postgres

**Deployment:** Docker container, persistent volume.

**Databases:**

- assistant — application data (channel_accounts, events, followups, pending_approvals, subscriptions_cursor)
- langgraph — LangGraph checkpointer tables
- langfuse — Langfuse tables

### Langfuse

**Purpose:** observability — every LLM call, every tool call, every agent invocation is a trace.

**Deployment:** Docker, self-hosted (not cloud), with its own Postgres.

### LLM router (deferred)

LiteLLM not included in v0.1. With one agent, direct use of LangChain's ChatOllama and ChatAnthropic is simpler. Add LiteLLM when 3+ agents exist and per-agent model choice becomes a config concern.

# 5. Data model

## 5.1 channel_accounts

```
CREATE TABLE channel_accounts (
  account_id    TEXT PRIMARY KEY,
  channel_type  TEXT NOT NULL,
  display_name  TEXT NOT NULL,
  credentials   JSONB NOT NULL,
  status        TEXT NOT NULL DEFAULT 'active',
  metadata      JSONB DEFAULT '{}',
  created_at    TIMESTAMPTZ DEFAULT now(),
  updated_at    TIMESTAMPTZ DEFAULT now()
);
```

Credentials stored as JSONB — OAuth tokens for Gmail, Baileys session reference for WhatsApp. Encryption at rest deferred to v0.2.

## 5.2 events

```
CREATE TABLE events (
  event_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  topic        TEXT NOT NULL,
  account_id   TEXT REFERENCES channel_accounts(account_id),
  payload      JSONB NOT NULL,
  published_at TIMESTAMPTZ DEFAULT now()
);
CREATE INDEX ON events(topic, published_at);
```

## 5.3 subscription_cursors

```
CREATE TABLE subscription_cursors (
  subscriber_id  TEXT NOT NULL,
```

```
  topic_pattern  TEXT NOT NULL,
  last_event_id  UUID,
  updated_at     TIMESTAMPTZ DEFAULT now(),
  PRIMARY KEY (subscriber_id, topic_pattern)
);
```

Each agent subscription tracks its position in the event log. Enables resumption after crash and is the conceptual match for NATS durable consumers.

### 5.4 followups

```
CREATE TABLE followups (
  followup_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  account_id   TEXT REFERENCES channel_accounts(account_id),
  thread_id    TEXT NOT NULL,
  direction    TEXT NOT NULL,
  deadline     TIMESTAMPTZ NOT NULL,
  note         TEXT,
  status       TEXT NOT NULL DEFAULT 'open',
  created_at   TIMESTAMPTZ DEFAULT now(),
  resolved_at  TIMESTAMPTZ
);
```

### 5.5 pending_approvals

```
CREATE TABLE pending_approvals (
  approval_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  agent_name    TEXT NOT NULL,
  action_type   TEXT NOT NULL,
  context       JSONB NOT NULL,
  status        TEXT NOT NULL DEFAULT 'pending',
  created_at    TIMESTAMPTZ DEFAULT now(),
  resolved_at   TIMESTAMPTZ,
  resolution    JSONB
);
```

Reserved for v0.2 when agents start requesting explicit approval for actions. v0.1 agents can only draft/label, which doesn't require this table — but the table exists so the protocol is ready.

## 6. Event schemas

### 6.1 EmailReceivedEvent

**Topic:** email.received.<account_id>

**Payload:**

```
{
  "event_id": "uuid",
```

```
  "ts": "2026-04-22T12:00:00Z",
  "source": "gmail",
  "account_id": "gmail.atheer",
  "message_id": "<gmail message id>",
  "thread_id": "<gmail thread id>",
  "from": "name <addr>",
  "to": ["..."],
  "cc": ["..."],
  "subject": "...",
  "body_text": "...",
  "body_html": "...",
  "snippet": "...",
  "labels": ["INBOX", "UNREAD"],
  "in_reply_to": null,
  "raw_ref": "gmail:messages/<id>"
}
```

## 6.2 WhatsAppMessageEvent

**Topic:** whatsapp.message.received.<account_id>

**Payload:**

```
{
  "event_id": "uuid",
  "ts": "...",
  "source": "whatsapp",
  "account_id": "wa.main",
  "msg_id": "<baileys id>",
  "chat_id": "<jid>",
  "from": { "jid": "...", "push_name": "..." },
  "is_group": false,
  "group_id": null,
  "text": "...",
  "media_ref": null,
  "quoted_msg_id": null,
  "raw_ref": "whatsapp:msg/<id>"
}
```

## 6.3 Topic design rules

- Hierarchy: <channel>.<event_type>.<account_id>. The account_id is always the most specific segment — enables per-account subscription without parsing.
- Wildcards: email.received.* (all accounts), email.*.gmail.atheer (all events for one account), *.received.* (all incoming everywhere).
- Backward compatibility: never remove fields from payloads. Add only. Agents must tolerate unknown fields.

# 7. Repository

## 7.1 Single monorepo with uv workspaces

One Git repository. All services, the shared package, and infrastructure configuration live together. Rationale: one user, shared schemas are the primary integration surface, no org/permissions story to justify splits.

## 7.2 Layout

```
assistant/
├── compose.yaml                  # all services in one file
├── .env                          # secrets (gitignored)
├── pyproject.toml                # uv workspace root
├── shared/
│   ├── pyproject.toml
│   └── src/shared/
│       ├── events.py
│       ├── approvals.py
│       ├── accounts.py
│       ├── bus.py
│       └── db.py
├── channels/
│   ├── gmail/
│   │   ├── pyproject.toml
│   │   ├── Dockerfile
│   │   └── src/gmail_adapter/
│   │       ├── main.py
│   │       ├── add_account.py
│   │       └── watch_renew.py
│   └── whatsapp/
│       ├── package.json
│       ├── Dockerfile
│       └── src/
│           ├── index.ts
│           └── add_account.ts
├── agents/
│   ├── pyproject.toml
│   ├── Dockerfile
│   └── src/agents/
│       ├── runtime.py
│       ├── registry.yaml
│       ├── tool_factory.py
│       └── email_pm/
│           ├── agent.py
```

```
│           ├── tools.py
│           └── prompts.py
├── resources/
│   └── migrations/
│       └── 001_init.sql
└── README.md
```

### 7.3 Service inventory

| Service | Where it runs | Purpose |
|---|---|---|
| postgres | Docker | Application DB, LangGraph checkpoints, Langfuse backend |
| langfuse | Docker | Observability traces |
| langfuse-db | Docker | Langfuse's own Postgres (keeps Langfuse upgrades isolated) |
| gmail-adapter | Docker | Gmail push + multi-account |
| whatsapp-adapter | Docker | Baileys sockets + multi-account |
| agents | Docker (or local for dev) | Agent runtime hosting all agents |
| ollama | Host (not Docker) | Local LLM serving Qwen3-30B |

## 8. Build plan

### 8.1 v0.1 milestone: Email PM end-to-end + WhatsApp pipe proven

**Day 1 (today): skeleton and infrastructure**

1. Initialize monorepo, uv workspace, compose.yaml with postgres + langfuse. [30 min]
2. Create shared package with event schemas, DB helper, bus abstraction. [45 min]
3. Apply 001_init.sql migration with all tables. [15 min]
4. Install and verify Ollama + Qwen3-30B-A3B-Instruct on host. [20 min + model pull]
5. Wire up Langfuse, verify traces land. [15 min]

**Day 2: Gmail adapter**

6. Google Cloud project, enable Gmail API, create Pub/Sub topic. [30 min]
7. Expose webhook endpoint via Tailscale Funnel. [15 min]
8. Implement add_account CLI with OAuth flow. [90 min]
9. Implement /gmail/push webhook — decode, history fetch, publish event. [90 min]
10. Onboard gmail.personal account. Verify events arrive. [30 min]

**Day 3: WhatsApp adapter**

11. Set up Node.js project with Baileys. [30 min]

12. Implement add_account CLI (QR scan). [60 min]
13. Implement messages.upsert handler + bus publish. [45 min]
14. Implement /whatsapp/send endpoint. [30 min]
15. Onboard wa.main. Verify events arrive and manual send works. [30 min]

**Day 4: Email PM agent**

16. Implement agent runtime with registry.yaml, subscriber loop. [90 min]
17. Build tool_factory.py for account-aware LangChain GmailToolkit. [60 min]
18. Implement custom tools: log_followup, list_overdue_followups, categorize_thread, escalate_to_claude. [90 min]
19. Write Email PM agent.py with LangGraph create_agent, system prompt, tool list. [90 min]
20. End-to-end test: send email to gmail.personal, verify label applied and draft created where appropriate. [60 min]

**Day 5: hardening and second accounts**

21. Onboard additional Gmail accounts (gmail.atheer, etc.). [15 min each]
22. Onboard wa.secondary. [15 min]
23. Verify multi-account isolation: Atheer email never drafted from personal.
24. Set up daily cron for overdue followup review.
25. Write README with setup, add-account, and 'add a new agent' walkthroughs.

### 8.2 v0.2 targets (next)

- Interface agent via Telegram bot (handles 'draft a reply to X', 'what's overdue?').
- First subject agent — Berwyn Heights deal — demonstrating that adding a subject agent is a 2-hour job using the existing bus and tools.
- LiteLLM introduced as resource when third agent ships.
- pending_approvals table put into real use (Telegram inline buttons for approve/reject).

### 8.3 v0.3 targets

- Calendar adapter (same pattern as Gmail — Google Pub/Sub push).
- Cross-channel subject agents (subject agent subscribes to email + WhatsApp both, filtered by relevance).
- Migration from Postgres-queue bus to NATS JetStream when agent count warrants.

## 9. Growth: adding agents and tools cheaply

### 9.1 Adding a new agent

Expected cost for a new subject agent (e.g. a specific project or relationship): 1–3 hours, mostly prompt engineering and deciding which memory to seed.

26. Create agents/src/agents/<name>/ with agent.py, prompts.py, and optionally tools.py for name-specific tools.
27. Add an entry to registry.yaml declaring subscriptions, schedule, account scope, and default model.
28. Seed core memory (for Letta-style agents) or initial checkpoint state (for LangGraph agents).
29. Deploy by restarting the agents service. New agent is live.

## 9.2 Adding a new channel

Expected cost: 1 day for a well-documented API like Slack or Calendar; 2–3 days for something exotic.

30. Create channels/<name>/ adapter service.
31. Define event schema(s) in shared/events.py.
32. Implement push → bus publish (prefer webhooks over polling).
33. Implement action API (or use LangChain toolkit if available).
34. Implement add_account CLI.
35. Onboard first account.

Existing agents that want to act on the new channel get new tools bound; existing agents that don't care are unaffected.

## 9.3 Adding a new tool (non-channel)

Examples: web search, Notion, filesystem, shell execution. Expected cost: under an hour.

36. Implement the tool as a LangChain BaseTool subclass (or use an existing toolkit).
37. Add to shared/tools/ (create this when first non-channel tool ships).
38. Bind to agents that need it via registry.yaml or agent config.

## 9.4 What reduces cost over time

- Shared schemas mean new adapters immediately interoperate with all existing agents.
- Bus abstraction means new agents can subscribe to any existing event type without touching the publisher.
- Account-aware tool factory pattern means adding an account is a DB row, not a code change.
- Agent runtime crash isolation means one bad agent doesn't break the others — lowers the risk of experiments.
- Langfuse traces mean debugging a weird agent behavior doesn't require print-statement archaeology.

## 9.5 What would increase cost (avoid these)

- Channel-specific logic leaking into agent code. Agents should never know Gmail's history_id or Baileys' JID format — that's what events hide.

- Implicit model routing. When an agent's model changes without the agent knowing, debugging becomes impossible. Keep escalation explicit.
- Shared mutable state across agents. Each agent owns its memory; cross-agent state is mediated by the bus, not by shared DB tables.
- Frameworks that require one-process-per-agent-type prematurely. v0.1's single-process runtime is a feature for now; upgrade only when needed.

## 10. Risks and mitigations

| Risk | Likelihood | Mitigation |
|---|---|---|
| Agent drafts wrong reply (wrong tone, misreads intent) | High | Draft-only, no autonomous send. Sol reviews every draft. Escalation to Claude for sensitive drafts. |
| Agent crosses accounts (replies from wrong email) | Medium | Account-parameterized tools + hard constraint in prompt + per-agent account scoping in registry. |
| WhatsApp Baileys ban | Low | Accepted for personal numbers. Business numbers use Cloud API when added. |
| Local model insufficient for some tasks | Medium | Explicit escalate_to_claude tool. Observability via Langfuse to see which tasks escalate often. |
| OAuth token expiry or revocation | Medium | Token refresh on 401, status column flips to 'broken', daily cron alerts Sol. |
| Bus contention / slow agents block others | Low (v0.1 scale) | Each subscription runs in its own async task. If it becomes a real issue, move to NATS + separate processes. |
| Prompt injection via email or WhatsApp content | Medium | Draft-only constraint is the primary defense. No tool in the agent's toolkit can cause real-world harm without Sol's approval. |
| Accidental deletion or modification of real Gmail data | Low | Agent has no delete tool. Labels are reversible. Drafts live in Drafts folder, never sent. |

## 11. Open decisions for Sol

39. OAuth scope for Gmail: gmail.modify (read + write labels + drafts + send) vs narrower. Recommended: gmail.modify, with send tool simply not bound to any agent.
40. WhatsApp accounts: confirm both numbers are personal (Baileys). Any business number here requires separate adapter path.
41. Webhook ingress: Tailscale Funnel vs ngrok vs Cloudflare Tunnel. Recommended: Tailscale Funnel (you likely already use Tailscale).
42. Which Gmail account to onboard first? Recommended: personal — highest volume, safest to test on.
43. Where should v0.1 'notifications' go? Options: Gmail labels only (current plan), email to yourself, or skip ahead and build a minimal Telegram notifier on Day 5. Recommended: labels only for v0.1; Telegram in v0.2.

44. Model escalation criteria for Email PM: agent-decided (explicit tool call) vs rule-based (e.g. always Claude for replies to VIPs). Recommended: agent-decided for v0.1, add rules after observing a week of traces.

## Appendix A: Naming conventions

- Account IDs: <channel>.<slug> — gmail.personal, gmail.atheer, wa.main.
- Topics: <channel>.<event_type>.<account_id>. Lowercase, dots as separators.
- Gmail labels applied by agents: pm/* prefix. Easy to filter and remove.
- Agent names: snake_case, matches directory name. email_pm, berwyn_deal.

## Appendix B: What this plan deliberately defers

- Encryption at rest for credentials — v0.1 relies on filesystem permissions. Upgrade: libsodium secretbox per-account.
- Multi-user / multi-machine deployment. v0.1 is one user, one machine.
- Backups. Postgres dump on cron is a day-6 task.
- Rate limiting of outbound actions. Trivial at current volume.
- End-to-end tests. Manual E2E only for v0.1. Automated tests arrive when agent count passes 3.
- A proper frontend. CLI + Gmail + (later) Telegram is the UI.